

Day 33 — Student Performance & Learning Analytics

Notebook export · Veda Technology

Day 33 — Student Performance & Learning Analytics

Veda Technology · Project 3 of 3

Analyze student academic data to identify performance patterns, subject-wise strengths, attendance trends, and factors associated with successful outcomes. This notebook uses synthetic data; no download is required.

1. Import Libraries

Code cell

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
sns.set_theme(style="whitegrid")
```

2. Generate Student Dataset

Code cell

```
np.random.seed(33)
n = 300
df = pd.DataFrame({
    "student_id": range(1, n+1),
    "attendance_pct": np.clip(np.random.normal(81, 10, n), 35, 100),
    "study_hours_daily": np.clip(np.random.normal(3.1, 1.3, n), 0.2, 9),
    "assignment_score": np.clip(np.random.normal(74, 12, n), 15, 100),
    "previous_score": np.clip(np.random.normal(69, 14, n), 10, 100),
    "math_score": np.clip(np.random.normal(70, 13, n), 10, 100),
    "science_score": np.clip(np.random.normal(72, 12, n), 10, 100),
    "english_score": np.clip(np.random.normal(73, 11, n), 10, 100)
})
df["final_score"] = np.clip(
    .18*df.attendance_pct + 2.4*df.study_hours_daily +
    .22*df.assignment_score + .20*df.previous_score +
    .12*df.math_score + .10*df.science_score +
    .08*df.english_score - 38 + np.random.normal(0, 7, n), 0, 100)
df["passed"] = (df.final_score >= 50).astype(int)
# Introduce missing values and duplicates for cleaning practice.
for col in ["attendance_pct", "study_hours_daily", "assignment_score"]:
    df.loc[np.random.choice(df.index, 4, replace=False), col] = np.nan
df = pd.concat([df, df.iloc[[0, 1]]], ignore_index=True)
df.head()
```

3. Inspect and Clean Data

Code cell

```
print("Original shape:", df.shape)
display(df.isna().sum().to_frame("missing_values"))
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates().copy()
numeric_cols = df.select_dtypes(include="number").columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
print("Cleaned shape:", df.shape)
display(df.describe().round(2))
```

4. Exploratory Data Analysis

Code cell

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(data=df, x="final_score", bins=20, kde=True, ax=axes[0])
axes[0].set_title("Distribution of Final Scores")
sns.scatterplot(data=df, x="attendance_pct", y="final_score", ax=axes[1])
axes[1].set_title("Attendance vs Final Score")
plt.tight_layout()
plt.show()
sns.scatterplot(data=df, x="study_hours_daily", y="final_score")
plt.title("Study Hours vs Final Score")
plt.show()
```

5. Subject-Wise Performance

Code cell

```
subjects = ["math_score", "science_score", "english_score"]
summary = df[subjects].agg(["mean", "median", "min", "max"]).T.round(2)
summary.columns = ["mean", "median", "minimum", "maximum"]
display(summary)
summary["mean"].sort_values().plot(kind="barh", title="Average Score by Subject")
plt.xlabel("Average score")
plt.show()
```

6. Attendance Analysis

Code cell

```
df["attendance_band"] = pd.cut(
    df.attendance_pct, bins=[0, 60, 75, 90, 100],
    labels=["Below 60%", "60-75%", "75-90%", "90-100%"],
    include_lowest=True)
attendance_summary = df.groupby("attendance_band", observed=False).agg(
```

```

student_count=("student_id", "count"),
average_attendance=("attendance_pct", "mean"),
average_final_score=("final_score", "mean"),
pass_rate=("passed", "mean")).round(2)
attendance_summary["pass_rate_pct"] = attendance_summary.pop("pass_rate") * 100
display(attendance_summary)

```

7. Correlation Analysis

Code cell

```

corr = df.select_dtypes(include="number").drop(columns="student_id").corr()
plt.figure(figsize=(10, 7))
sns.heatmap(corr, cmap="coolwarm", center=0, annot=True, fmt=".2f")
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
display(corr.final_score.sort_values(ascending=False).to_frame("correlation_with_final_score"))

```

8. Feature Engineering

Code cell

```

df["average_subject_score"] = df[subjects].mean(axis=1)
df["assignment_previous_average"] = df[["assignment_score", "previous_score"]].mean(axis=1)
df["attendance_study_index"] = df.attendance_pct * df.study_hours_daily
features = ["attendance_pct", "study_hours_daily", "assignment_score",
            "previous_score", "math_score", "science_score", "english_score",
            "average_subject_score", "assignment_previous_average",
            "attendance_study_index"]
display(df[features + ["final_score", "passed"]].head())

```

9. Linear Regression — Final Score Prediction

Code cell

```

X, y = df[features], df.final_score
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=33)
reg = LinearRegression().fit(X_train, y_train)
pred = reg.predict(X_test)
print("MAE:", round(mean_absolute_error(y_test, pred), 2))
print("RMSE:", round(np.sqrt(mean_squared_error(y_test, pred)), 2))
print("R²:", round(r2_score(y_test, pred), 3))

```

10. Logistic Regression — Pass/Fail Classification

Code cell

```

Xtr, Xte, ytr, yte = train_test_split(
    X, df.passed, test_size=0.2, random_state=33, stratify=df.passed)
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
clf.fit(Xtr, ytr)
pred_class = clf.predict(Xte)
print("Accuracy:", round(accuracy_score(yte, pred_class), 3))
print(classification_report(yte, pred_class, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(yte, pred_class))

```

11. Model Interpretation

Code cell

```

coef = pd.Series(reg.coef_, index=features).sort_values(
    key=lambda x: x.abs(), ascending=False)
display(coef.to_frame("linear_regression_coefficient"))
coef.sort_values().plot(kind="barh", figsize=(8, 5), title="Regression Coefficients")
plt.tight_layout()
plt.show()

```

12. Exploratory Student Support Review

Code cell

```

review = df[["student_id", "attendance_pct", "study_hours_daily",
            "average_subject_score", "final_score", "passed"]].copy()
review["review_flag"] = np.select(
    [(review.attendance_pct < 60) | (review.final_score < 40),
     (review.attendance_pct < 75) | (review.final_score < 50)],
    ["Review recommended", "Monitor"],
    default="No flag from these simple rules")
display(review.review_flag.value_counts().to_frame("student_count"))
display(review.sort_values("final_score").head(10))

```

13. Findings Template

After running the notebook, document:

- Data quality: initial/cleaned row counts and missing-value handling.
- Subject performance: highest and lowest average subjects.
- Attendance: compare average scores and pass rates by attendance band.
- Relationships: describe correlations without claiming causation.
- Regression: report MAE, RMSE, and R².
- Classification: report accuracy, precision, recall, F1-score, and confusion matrix.
- Limitations: synthetic data may not represent real students.

14. Interview Questions

- What is EDA?

- How do you handle missing values and duplicates?
- What is the difference between correlation and causation?
- Linear Regression vs Logistic Regression?
- Why split data into training and test sets?
- What do MAE, RMSE, and R^2 measure?
- What are precision, recall, F1-score, and a confusion matrix?
- Why is feature scaling useful?
- How would you explain findings to a non-technical audience?

****Responsible use:**** This project is educational. Synthetic data and simple review flags must not be used for real-world student decisions.